

INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY

INTRO TO CRITICAL INFRASTRUCTURE FOR IT LEARNERS

LAB 02 — PLC PROGRAMMING LANGUAGES

Student Edition

Authorized ICDFA training environments only
icdfa.edu.ng

Student Name	Pedro Miambo
Registration Number	C11/26/ACIS/17373
Signature	Pedro Miambo
Date	27-09-2026

Lab 02: PLC Programming Languages

Course: Intro to Critical Infrastructure for IT Learners | **Estimated Duration:** 2–3 hours

Submission deadline: As published in ICDFA E-Campus.

Authorization & Safety: Perform all activities only on the ICDFA-provided virtual machines, packet captures and isolated training networks. Do not bridge these VMs to production networks, scan external systems, or reuse credentials observed in training data.

Learning Outcomes

- Compare the PLC programming languages presented in the course.
- Interpret a motor start/stop latching circuit in Ladder Diagram.
- Recognize equivalent motor-control logic represented using Function Block Diagram, Structured Text, Instruction List and Sequential Function Chart.
- Explain how tags and Boolean data types support PLC logic.

Laboratory Environment

System	Expected Address	Account	Password
PLC VM	192.168.1.10	plc	password

Note: If the instructor-provided VM image uses updated addressing or credentials, use the values published in E-Campus. Do not alter addressing unless instructed.

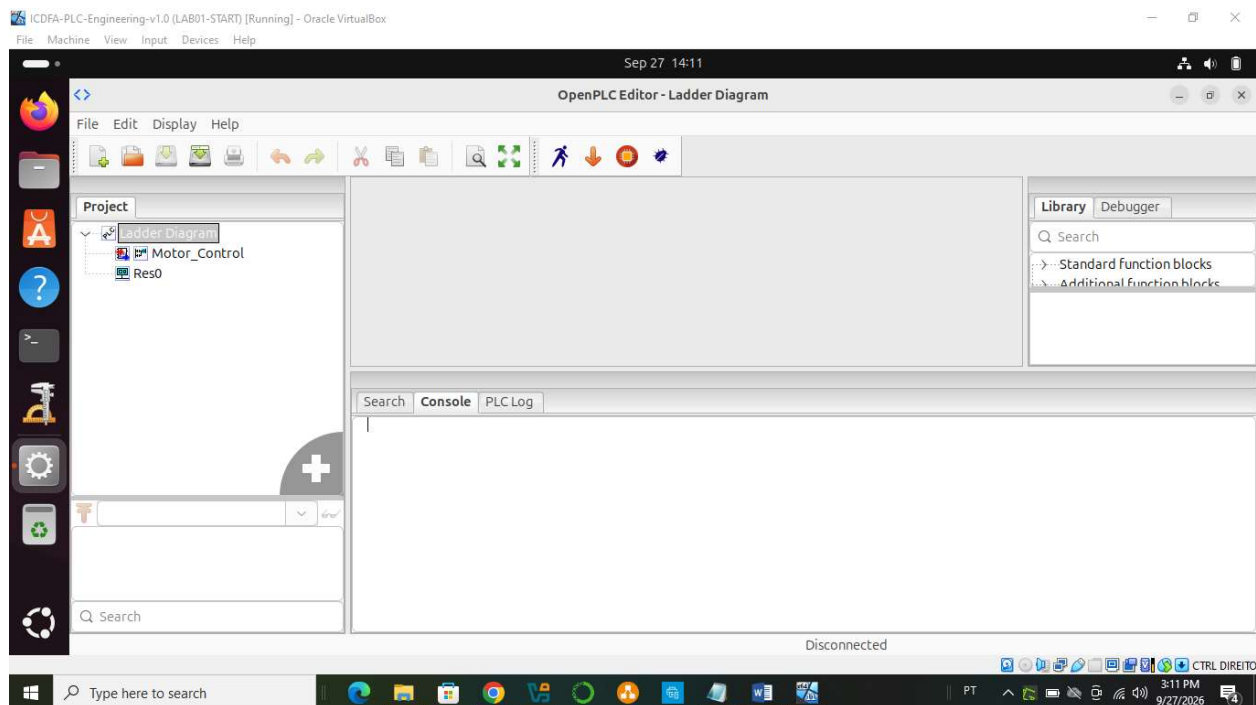
Pre-Lab Checklist

- Read the complete lab before beginning.
- Confirm you can access the required VM(s) and that virtualization support is enabled.
- Use the instructor-specified isolated/host-only network.
- Create or restore the clean starting snapshot for this lab.
- Prepare a report document with your name, registration number, course, lab number and date.
- Capture evidence as you progress rather than repeating the lab at the end.

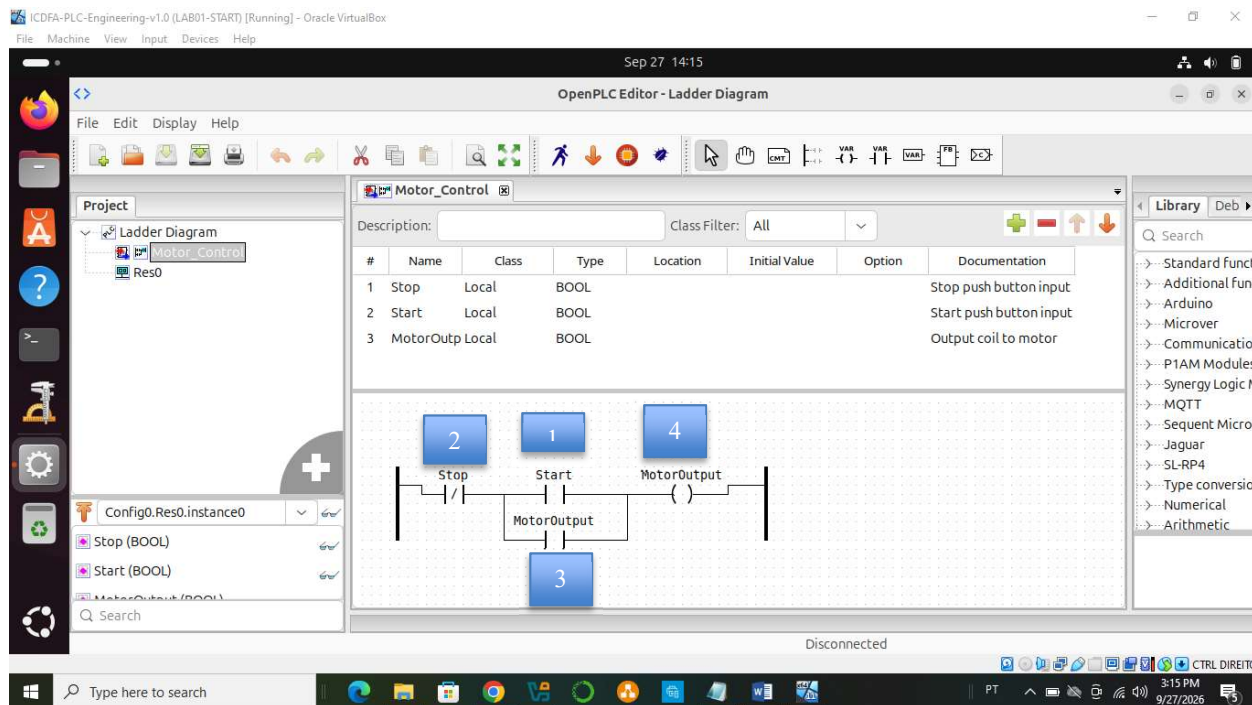
Procedure

Step 1. Start from the clean ICDFA PLC VM snapshot and sign in to the PLC training environment.

Step 2. Launch OpenPLC Editor and open the Ladder Diagram example from the PLC Programming Language Examples directory.

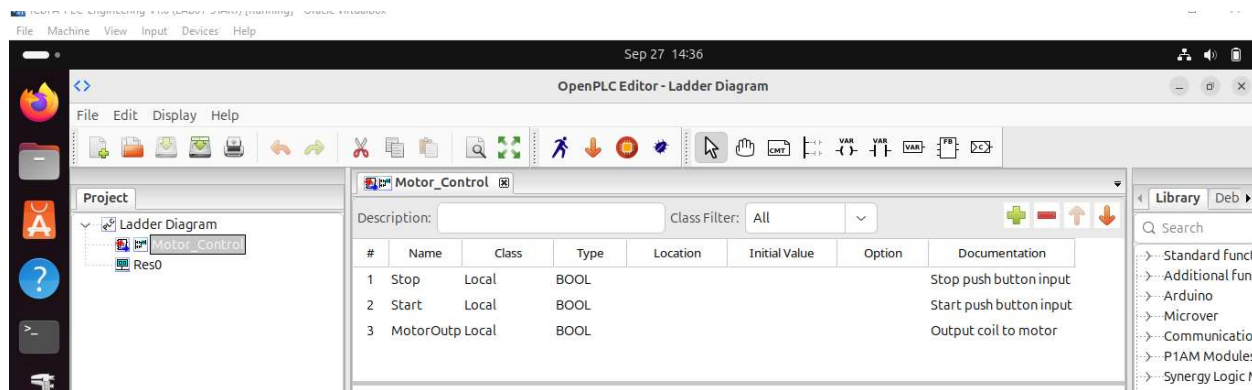


Step 3. Open Res0 > instance0 (Motor_Control). Identify the normally open Start contact, normally closed Stop contact, MotorOutput latch contact and MotorOutput coil.



- (1) – NO Start contact
- (2) – NC Stop contact
- (3) – MotorOutput latch contact
- (4) – MotorOutput coil

Step 4. Open the tag/variable list. Confirm that the motor-control tags use Boolean values and record the purpose of each tag in your worksheet.

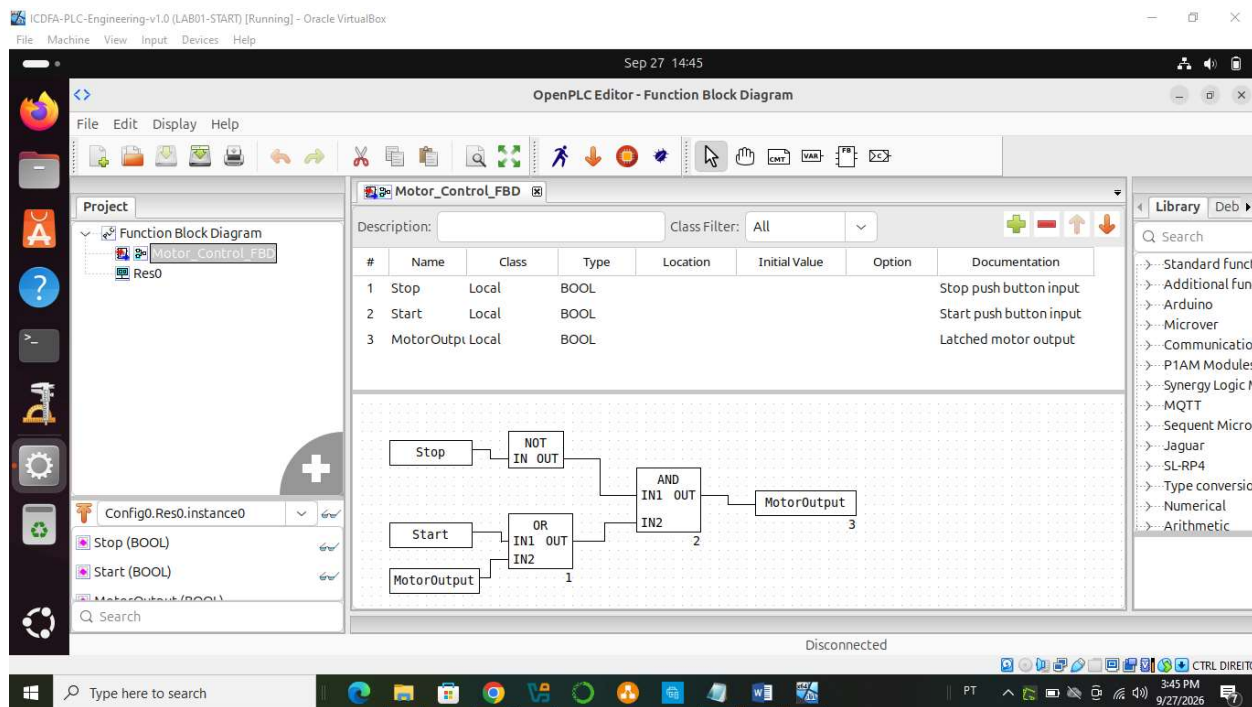


Tag Purpose Worksheet

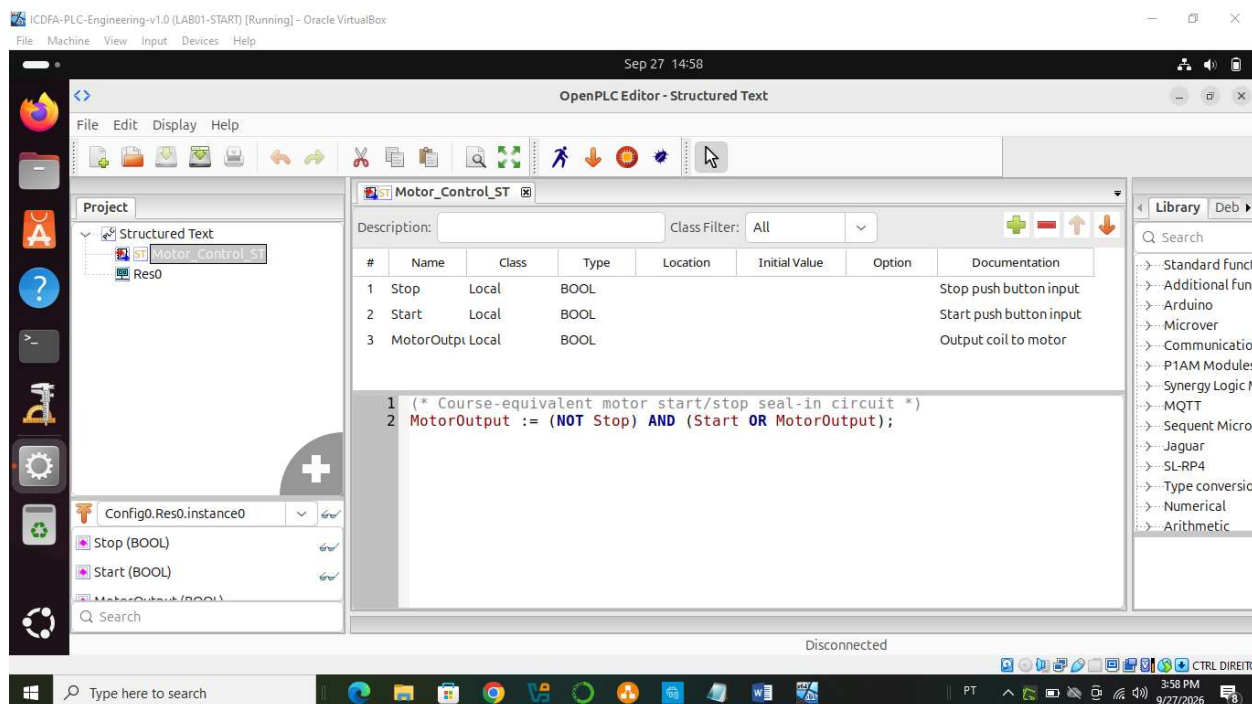
Tag	Purpose
Stop	Stop push button input
Start	Start push button input
MotorOutput	Output coil to motor

Step 5. Return to File > Open and load the Function Block Diagram example. Open Motor_Control_FBD and identify the block that maintains the motor output until a stop condition occurs.

A: OR block is the one that maintains the motor output until a stop condition occurs.



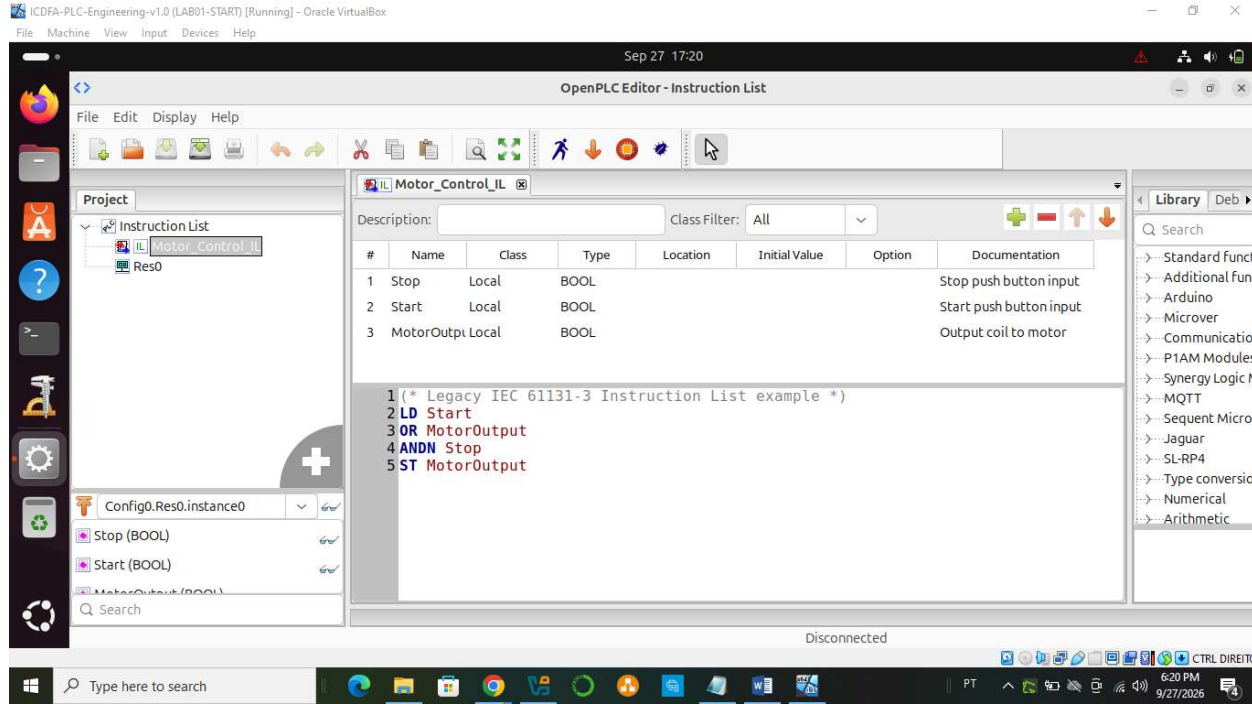
Step 6. Open the Structured Text example and inspect MotorControlST. Describe how the same start/stop behavior is represented using text rather than graphical contacts and coils.



✓ The Output Coil (:=) - A standard output coil at the right end of a ladder rung.

- ✓ The Normally Closed Stop Button (NOT Stop) - A normally closed (NC) contact [/] in series with the rest of the circuit. It passes power when it is *not* pressed.
- ✓ The Start Button and Latching Circuit (Start OR MotorOutput) - A normally open (NO) Start contact [] wired in parallel with a NO MotorOutput sealing/latching contact.
- ✓ The Series Connection (AND) - Interconnecting components horizontally in a line to form a continuous path for power.

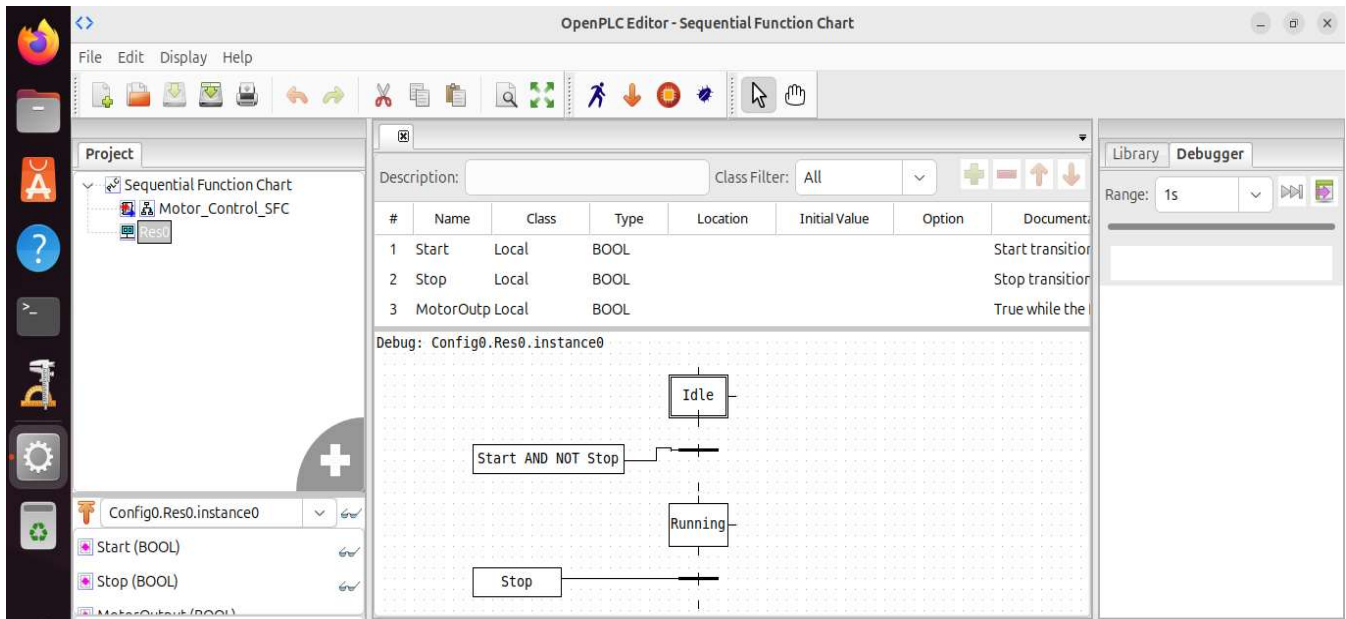
Step 7. Open the Instruction List example and inspect MotorControl. Record the main visual difference between this low-level textual representation and Structured Text.



A: The main visual difference is that the Instruction List (IL) code is organized as a vertical, line-by-line sequence of low-level reserved keywords (like LD, OR, ANDN, and ST), whereas Structured Text (ST) uses an algebraic block-structured syntax that resembles high-level programming languages like Python.

Visual Feature	Instruction List (IL)	Structured Text (ST)
Code Layout	One short mnemonic instruction and operand per line in a strict vertical list.	Long, continuous algebraic expressions grouped into logical blocks.
Variable Assignment	Uses accumulator operations like LD (Load) to start and ST (Store) to save.	Uses the standard assignment operator := (e.g., MotorOutput := ...).
Logic Representation	Logical operations are stacked vertically (e.g., OR, ANDN).	Logic is expressed inline using standard operators (e.g., AND NOT, OR).

Step 8. Open the Sequential Function Chart example and inspect MotorControl_SFC. Identify how steps/transitions express process sequence.



In the **Motor_Control_SFC Sequential Function Chart**, the process sequence is expressed through a structured, top-down flow alternating between states and conditional triggers:

Steps (States): These represent the status of the system.

- ✓ Idle: The initial step, marked by a double border, meaning the system starts here and waits in a standby state.
- ✓ Running: The active step that executes when the motor is operational.
- ✓ Transitions (Triggers): These are the horizontal lines containing logical conditions that must be met to move from one-step to the next.
 - **Start AND NOT Stop**: The condition required to transition from Idle to Running.
 - **Stop**: The condition required to exit the Running step.

Process Sequence Flow

The execution sequence flows strictly downwards. The system activates the **Idle** step first. It continuously checks the underlying transition; once **Start** is true AND **NOT Stop** is true, the chart deactivates **Idle** and activates **Running**. The system will hold in the **Running** state until the **Stop** transition condition becomes true.

Step 9. Complete the language comparison matrix in the lab report for all five representations.

Language	Representation	Key Strengths	Core Limitations	Suitable Industrial Use Case
Ladder Diagram (LD)	Graphical; mimics traditional hardwired electrical relay logic diagrams with horizontal rungs and vertical power rails.	Highly intuitive for electricians; excellent for discrete Boolean logic and rapid, real-time visual troubleshooting.	Extremely bulky for mathematical equations; poor handling of data loops (FOR/WHILE).	Conveyor system control, safety interlock logic, and factory-floor motor control.
Function Block Diagram (FBD)	Graphical; maps process signals left-to-right through interconnected modular boxes resembling logic gates.	Exceptional for continuous control loops; allows seamless data feedback without intermediate tracking variables.	Demands significant screen space; becomes messy, cluttered, and unreadable if blocks grow too complex.	PID temperature regulation, chemical flow control loops, and water treatment systems.
Structured Text (ST)	Textual; high-level structured language with syntax rules heavily resembling Pascal, C, or Python.	Native support for nested conditional logic and iterations; 15–25% faster execution for intricate math.	Lacks real-time graphical visual feedback, making it much harder for plant technicians to quickly debug.	Complex data processing, array tracking, recipes manipulation, and advanced algorithmic math.
Instruction List (IL)	Textual; low-level assembly-like command list relying on simple mnemonics and hardware registers.	Highly compact; executes at high speed with minimal memory overhead on very low-tier controller platforms.	Steep learning curve; difficult to trace and officially deprecated in recent industry standard revisions.	Legacy system maintenance, high-speed hardware-interrupt routines, and micro-controllers.
Sequential Function Chart (SFC)	Graphical; visual flowchart architecture utilizing distinct organizational states (steps) and conditional transitions.	Simplifies complex, step-driven behaviors; isolates faults directly to a specific inactive phase.	Not ideal for raw logic; acts purely as an organizer and needs underlying languages to manage output actions.	Batch process manufacturing (e.g., breweries, paint blending) and automated machine startup cycles.

Step 10. Capture one evidence screenshot for each programming representation, ensuring the program name is visible.

A short explanation showing how the same motor start/stop requirement can be represented differently while preserving the same control objective.

Response: The same motor start/stop control logic can be implemented across all five PLC programming languages while preserving the same control objective: **turning a motor on with a Start button, latching it active (MotorOutput), and cutting power immediately if a Stop signal occurs.**

The core Boolean logic across all these representations is fundamentally the same:
 MotorOutput = (Start OR MotorOutput) AND (NOT Stop)

Required Evidence & Submission

- Five screenshots: Ladder Diagram, Function Block Diagram, Structured Text, Instruction List and Sequential Function Chart.
- A completed comparison matrix covering representation, strengths, limitations and a suitable industrial use case.
- A short explanation showing how the same motor start/stop requirement can be represented differently while preserving the same control objective.

Recommended filename: <RegistrationNo>_LAB02_PLC_Programming_Languages.pdf

Knowledge Check

1. Why is Ladder Diagram intuitive for technicians familiar with relay logic?

Response: Ladder Diagrams are highly intuitive for technicians because their visual format is similar to old relay logic. Knowing how to read electrical schematic, moving to digital code becomes easy due to key similarities:

- ✓ *The layout is identical* - The software layout looks a traditional wiring diagram
- ✓ Same old symbols: Contacts and coils use the same symbols as physical relays.
- ✓ Electricity flow: The code reads from left to right, which mimics the way current actually moves through a circuit
- ✓ Easy troubleshooting: Finding a fault is simple because it is just tracing the path visually, similar to testing live wires on a real control panel using a multimeter.

2. What is one advantage of Structured Text for complex calculations or conditions?

Response: One major advantage of Structured Text (ST) for complex calculations or conditions is its native support for high-level programming constructs, such as loops (FOR, WHILE) and nested conditional statements (IF-THEN-ELSE, CASE). Unlike graphical PLC languages, which become difficult to read when dealing with complex calculations or conditions, Structured Text uses a text-based syntax similar to languages like Python.

3. When might a Sequential Function Chart be more appropriate than a single ladder rung?

Response: Sequential Function Chart is more appropriate than a single ladder rung in circumstances of batch manufacturing and recipe execution.

4. Why must the meaning of tags remain consistent when the representation/language changes?

Response: Systems cannot talk to each other without consistent tags. Tags are just the labels that explain data to software. If a label changes meaning between formats or languages, the system breaks. Keeping them uniform is what keeps the whole operation running smoothly.

Marking Rubric — 100 Marks

Criterion	Marks	Standard
Technical execution	50	Lab procedure completed correctly; required configuration/analysis works as intended.
Evidence	20	Screenshots/files are complete, readable, relevant and clearly labelled.
Analysis & knowledge check	20	Responses demonstrate understanding of the industrial-control concepts observed.
Professional reporting	10	Clear structure, correct terminology, concise writing, file naming and submission compliance.

Submission Integrity: Evidence must come from the student's own assigned laboratory environment. Screenshots should show enough context to verify the activity. Do not submit another learner's screenshots or project files.